

Performance Testing

Date	1 October 2026
Team ID	6ab501466f290be8a6b4f5d3
Project Name	Comic Craft AI App
Maximum Marks	5 Marks

Performance Testing

Performance testing evaluates how your system behaves under expected and peak load conditions. This report documents the testing approach, tools used, and results obtained for the Comic Craft AI App.

Step 1: Testing Overview

Field	Details
Testing Tool Used	Locust 2.46.6 (Python) for load generation; psutil for CPU and memory sampling; cProfile for bottleneck analysis.
Type of Testing	Load Testing at expected concurrency (50 / 25 / 10 / 3 virtual users), plus a before-and-after performance comparison of one optimization.
Target Module / API	Home page (GET /), health check (GET /health), static CSS, story API (POST /generate-comic/json), image API (POST /generate-comic/images) and the full comic flow with PDF export (POST /generate).
Test Environment	Local: Linux, 1 vCPU, about 3.9 GB RAM, Python 3.12, FastAPI on Uvicorn (1 worker). Locust ran on the same machine. Story provider stubbed (no Gemini API key, no network latency); local image provider (project default); real PDF export.
Test Date	2 October 2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Home page browsing: GET /, GET /health and GET /static/css/styles.css, 0.5 to 1.5 s think time	50	30	No errors; pages respond in well under 2 s.
2	Story generation API: POST /generate-comic/json with varied prompts, characters, settings, tones and art styles (story provider stubbed)	25	30	Average under 2 s; error rate under 1%.
3	Panel image generation: POST /generate-comic/images, five local images per request	10	30	Average under 2 s; error rate under 1%.
4	Full comic generation: POST /generate (story, five images, PDF export, preview page)	3	60	Average under 2 s, maximum under 5 s; no errors.

Step 3: Performance Test Results

Metrics are for the core user journey, full comic generation (Scenario 4, 3 virtual users), shown as baseline (repository code as is) → optimized (font-cache fix). Per-scenario figures follow in the second table.

S.No	Metric	Target Value	Actual Value (Baseline → Optimized)	Status (Pass / Fail)	Remarks
1	Response Time (Avg)	< 2 seconds	22.16 s → 0.47 s	Fail → Pass	Baseline slowed by PDF font build; 47x faster after the font-cache fix.
2	Response Time (Max)	< 5 seconds	23.87 s → 0.95 s	Fail → Pass	95th percentile: 24.0 s → 0.79 s.
3	Throughput (Req/sec)	≥ 1 req/sec (assumed, 3 users)	0.13 → 1.55 req/sec	Fail → Pass	About 12x higher throughput for the full comic flow.
4	Error Rate	< 1%	0% → 0%	Pass	0 failures in 6 baseline and 93 optimized requests.
5	CPU Utilization	< 80%	94% → 43% (average)	Fail → Pass	Peak 100% → 89%; Locust shares the single vCPU.
6	Memory Utilization	< 80%	371 MB (16.8%) → 177 MB (11.7%) peak	Pass	Server RSS peak; percentage is of 3.9 GB system RAM.

Summary of all scenarios (Locust console totals, 0 failures in every run):

Scenario	Requests	Avg (ms)	p95 (ms)	Max (ms)	Req/sec	Failures
S1 Home page (50 users)	1,416	3	7	62	47.23	0
S2 Story JSON (25 users)	719	4	7	34	24.02	0
S3 Image generation (10 users)	263	162	320	496	8.78	0
S4 Full comic, baseline (3 users)	6	22,160	24,000	23,867	0.13	0
S4 Full comic, optimized (3 users)	93	474	790	945	1.55	0

Step 4: Observations & Analysis

Key Findings:

Static pages and the story API are very light. Home page, health check and CSS ran at 47 req/sec with a 3 ms average, and the story API at 24 req/sec with a 4 ms average, with zero failures. The story provider was stubbed, so real Gemini latency (network and model time) is not included and was not measured.

Local panel generation is moderate. Image generation averaged 162 ms (95th percentile 320 ms) at 10 concurrent users, with server CPU averaging 63% and peaking at 86%.

The full comic flow was the weak point. With only 3 concurrent users it averaged 22.2 s per comic and handled 0.13 req/sec, with the server CPU pinned near 100% and memory peaking at 371 MB. A single unloaded request already took about 8 s.

Bottlenecks Identified:

PDF export (services/pdf_export.py, _build_pdf) is the main bottleneck. Profiling with cProfile showed over 95% of export time inside fpdf2 add_font, which uses fontTools to rebuild the NotoSans variable font into a static weight (wght 400 and 700) on every export. The cost is CPU-bound, so concurrent exports queue on the CPU and response time grows with the number of users.

Secondary: image generation is also CPU-bound (Pillow drawing), but it stayed under 0.5 s in every request.

Optimization Steps Taken:

Prototype fix, applied to a separate copy of the code and not committed to the repository: the two static font weights (400 and 700) are generated once, cached, and reused by add_font, and the cache is warmed at startup. The exported PDF was compared with the original: same 5 pages, identical text and no visible rendering differences.

Result under the same 3-user load: average 22.16 s → 0.47 s, maximum 23.87 s → 0.95 s, throughput 0.13 → 1.55 req/sec, server CPU 94% → 43% average, peak memory 371 MB → 177 MB, still with 0 failures.

Recommended next steps: commit the font-cache change (or bundle pre-generated static Noto Sans Regular and Bold files), run PDF export outside the request thread if traffic grows, and repeat the test with the real Gemini provider and a multi-worker Uvicorn setup.

Assumptions and limitations:

Targets (2 s average, 5 s maximum, under 1% errors, under 80% CPU and memory) and the 1 req/sec throughput goal are assumed service levels, since the project defines none. Results are from a single-vCPU test machine that also ran the load generator, so absolute numbers are indicative; the before-and-after comparison is the reliable result. Virtual users used a 0.5 to 1.5 s think time (1 to 2 s for Scenario 4).

Step 5: Screenshots / Evidence

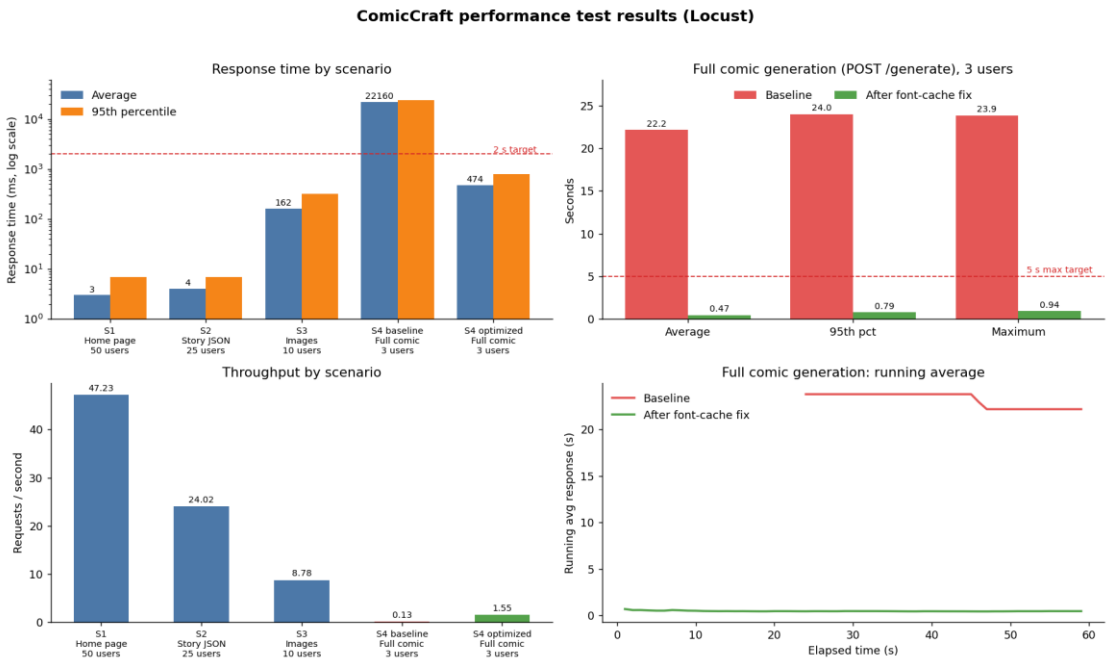
Screenshots of the performance testing results below (Locust output, response-time charts and resource utilization). Charts and tables were generated from the Locust CSV and console output and from psutil samples recorded during the test runs.

locust --headless --csv out/<scenario> --only-summary | ComicCraft @ http://127.0.0.1:8000

Type	Name	# reqs	# fails	Avg	Min	Max	Med	req/s
# Scenario 1								
GET	/	693	0(0.00%)	3	1	34	3	23.12
GET	/health	437	0(0.00%)	2	1	32	2	14.58
GET	/static/css/styles.css	286	0(0.00%)	4	2	62	3	9.54
# Scenario 2								
POST	/generate-comic/json	719	0(0.00%)	4	2	34	4	24.02
# Scenario 3								
POST	/generate-comic/images	263	0(0.00%)	162	59	496	140	8.78
# Scenario 4 (baseline)								
POST	/generate	6	0(0.00%)	22160	20334	23867	23867	0.13
# Scenario 4 (optimized)								
POST	/generate	93	0(0.00%)	474	239	945	450	1.55

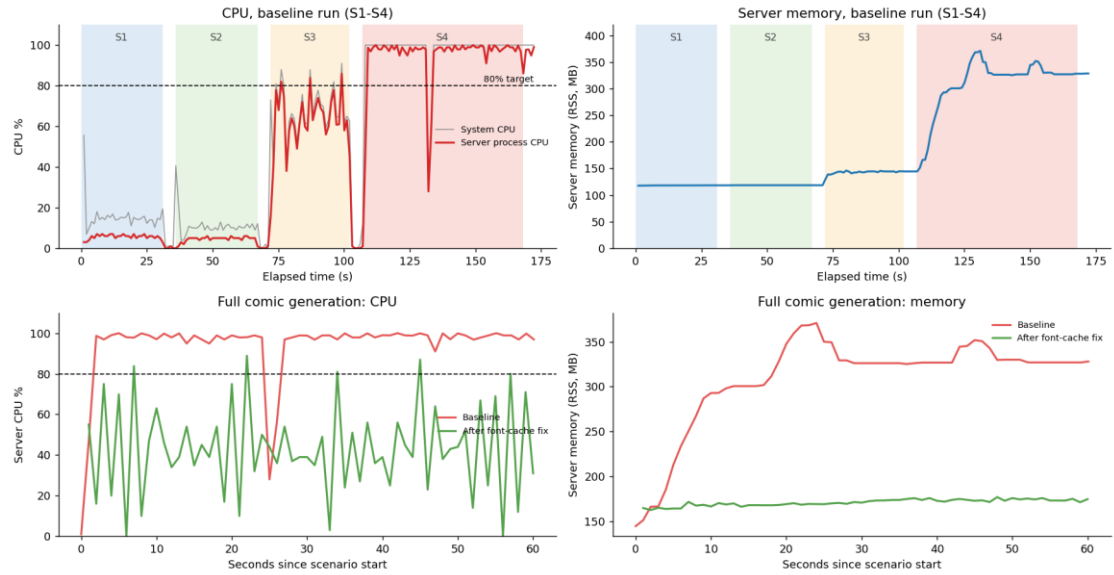
Response times in ms. All requests completed with 0 failures.

Screenshot 1: Locust statistics for all scenarios (requests, failures, response times in ms, requests per second)



Screenshot 2: Response time, full-comic before vs after the fix, throughput and running average

Resource utilisation during the test (psutil, sampled every second)



Screenshot 3: Server CPU and memory over the test run, and before vs after for full comic generation